



Flappy Bird Neural Network Simulation from Scratch

By Fatima Laraib

Github Link: <https://github.com/fatimaLaraibBonk/Flappy-Bird-Neural-Network-Simulation>

Flappy Bird Neural Network

Abstract

A Neural Network built from scratch that plays Flappy Bird.

Tools

Python

Libraries

NumPy, PyGame

Problem Case/ Problem Introduction

Flappy Bird is a 2 Dimensional game where the user plays as a bird that has to fly through obstacles whilst trying not to collide with any of the incoming structures. The bird automatically moves forward with its own velocity that can increase as time goes on. The player can tap/press a key to make the bird move upward, if the player does not press/tap anything the bird will glide down with a constant acceleration. Once the bird hits any of the incoming structures, the game is over.

To get a visual simulation of the game, visit: <https://flappybird.io/>

Solution Implemented

We will attempt to build a neural network from scratch that plays the game and optimizes its score. Before we move onto the code we must define some parameters that we will working under:

- Actions
- Constraints
- Game Termination Condition

Actions: The term “actions” means the possible steps/moves the neural network can make while playing the game. In our case the neural network can only simulate a player’s tap response, which allows the bird to move up. Hence the neural network has two possible choices at any moment: *Move Up or Do Not Move Up (Default)*.

Constraints: By constraints we mean what the Neural Network is not allowed to do while playing the game. The neural network *can not stop the bird or make the bird move backwards*.

Game Termination Condition: The game will end/terminate when the bird collides with one of the incoming structures.

Algorithm:

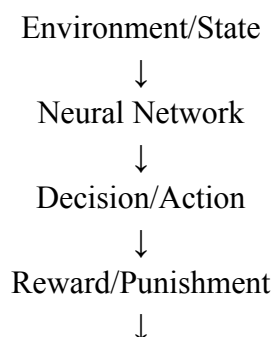
To solve this problem we will be use *Q-learning* (a type of reinforcement learning) for which we will have to further design:

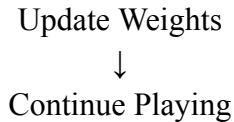
- State: The current state of the environment that our model can observe.
- Action: The decision/action our model decides to take such as Move Up or not.
- Reward/Penalty: Reward if it passes through the structure, penalty if it crashes and nothing if neither situation occurs.

Concept: In Q-learning, we follow an agent-learner pairing where the learner will be our Neural Network, and we will be the Agent, i.e. the one who will be training it by giving rewards/punishments.

Flow

A small visual to understand how exactly the solution is going to be implemented since we will be using multiple snippets of code





State: The information that our model needs to know to make a decision, in our case, the model needs to know the following: Bird's coordinates/Position, the bird's velocity, the incoming structure/pillars coordinates/position, the gap height. States can be binary/boolean (*One-Hot Encoding*) or they can be numeric. For each state, the learner./model will decide on an action to carry out.

Action: Based on the state, the model will decide on an action: Move Up or Not. An action will take the bird into a new state that can have a certain result: the bird can collide with the structure, the bird can pass through the gap in the structure, or still not be in contact with a structure. And each state and action pair has its respective reward depending on the result of the action.

Goal: Our goal is to maximize the reward.

$$\text{New } Q(S, A) = Q(S, A) + \alpha [R(S, A) + \gamma \max_{A'} Q(S', A') - Q(S, A)]$$

Reward Calculation: To define the rewards we need to assess each situation.

Did the bird collide with a structure? -10

If not, then did it pass through a gap in a structure? $+10$

If not then there is neither reward nor punishment.

Further down we will also look at the *Exploration vs Exploitation* concept.

Simulation Aspect: To visualise both the game and NN, we will be using pyGame.

Yellow circle: Bird

Green rectangles: Structures

Blue Background

Neural Network Basics

Activation Function

The activation function helps introduce non-linearity in the model. Basically in a linear model one value has a linear effect on the output, for example assume 'hours_studied' and the output 'score_achieved'. In a linear model the hours_studied will make a linear relationship with the output that can be represented by a line equation $y=mx+b$, however neural networks are for more complex relationships where hours_studied might not always cause output to move in a single direction.

There are many different neural networks such as ReLu, Sigmoid etc. For our project, we are using one of the most common ones, that is the ReLu activation function.

Consistency and Stability

Consistency:

Stability:

Exploration Vs Exploitation

Case Study

History to Focus on:

Episodes 1-10 | avg = -5.00 | best = 40 | epsilon = 0.951
Episodes 11-20 | avg = -5.00 | best = 40 | epsilon = 0.905
Episodes 21-30 | avg = -10.00 | best = -10 | epsilon = 0.860
Episodes 31-40 | avg = 10.00 | best = 90 | epsilon = 0.818
Episodes 41-50 | avg = 25.00 | best = 190 | epsilon = 0.778
Episodes 51-60 | avg = 65.00 | best = 540 | epsilon = 0.740
Episodes 61-70 | avg = 80.00 | best = 240 | epsilon = 0.704
Episodes 71-80 | avg = 55.00 | best = 190 | epsilon = 0.670
Episodes 81-90 | avg = 180.00 | best = 440 | epsilon = 0.637
Episodes 91-100 | avg = 150.00 | best = 540 | epsilon = 0.606
Episodes 101-110 | avg = 240.00 | best = 1240 | epsilon = 0.576
Episodes 111-120 | avg = 135.00 | best = 940 | epsilon = 0.548
Episodes 121-130 | avg = 170.00 | best = 890 | epsilon = 0.521
Episodes 131-140 | avg = 230.00 | best = 1090 | epsilon = 0.496
Episodes 141-150 | avg = 130.00 | best = 1240 | epsilon = 0.471
Episodes 151-160 | avg = -10.00 | best = -10 | epsilon = 0.448
Episodes 161-170 | avg = -5.00 | best = 40 | epsilon = 0.427
Episodes 171-180 | avg = 325.00 | best = 1840 | epsilon = 0.406
Episodes 181-190 | avg = 190.00 | best = 1940 | epsilon = 0.386
Episodes 191-200 | avg = 200.00 | best = 940 | epsilon = 0.367
Episodes 201-210 | avg = 100.00 | best = 290 | epsilon = 0.349
Episodes 211-220 | avg = 95.00 | best = 290 | epsilon = 0.332
Episodes 221-230 | avg = 35.00 | best = 190 | epsilon = 0.316
Episodes 231-240 | avg = 460.00 | best = 4290 | epsilon = 0.300
Episodes 241-250 | avg = -5.00 | best = 40 | epsilon = 0.286
Episodes 251-260 | avg = -10.00 | best = -10 | epsilon = 0.272
Episodes 261-270 | avg = -5.00 | best = 40 | epsilon = 0.258
Episodes 271-280 | avg = 15.00 | best = 140 | epsilon = 0.246
Episodes 281-290 | avg = 0.00 | best = 40 | epsilon = 0.234
Episodes 291-300 | avg = 240.00 | best = 1590 | epsilon = 0.222
Episodes 301-310 | avg = -10.00 | best = -10 | epsilon = 0.211
Episodes 311-320 | avg = -10.00 | best = -10 | epsilon = 0.201
Episodes 321-330 | avg = -10.00 | best = -10 | epsilon = 0.191
Episodes 331-340 | avg = -10.00 | best = -10 | epsilon = 0.182
Episodes 341-350 | avg = -10.00 | best = -10 | epsilon = 0.173
Episodes 351-360 | avg = -10.00 | best = -10 | epsilon = 0.165
Episodes 361-370 | avg = 5.00 | best = 140 | epsilon = 0.157
Episodes 371-380 | avg = -10.00 | best = -10 | epsilon = 0.149
Episodes 381-390 | avg = -10.00 | best = -10 | epsilon = 0.142
Episodes 391-400 | avg = -5.00 | best = 40 | epsilon = 0.135
Episodes 401-410 | avg = -10.00 | best = -10 | epsilon = 0.128
Episodes 411-420 | avg = 0.00 | best = 90 | epsilon = 0.122
Episodes 421-430 | avg = 110.00 | best = 1040 | epsilon = 0.116
Episodes 431-440 | avg = 15.00 | best = 90 | epsilon = 0.110
Episodes 441-450 | avg = -10.00 | best = -10 | epsilon = 0.105
Episodes 451-460 | avg = -10.00 | best = -10 | epsilon = 0.100
Episodes 461-470 | avg = -10.00 | best = -10 | epsilon = 0.095
Episodes 471-480 | avg = -10.00 | best = -10 | epsilon = 0.090
Episodes 481-490 | avg = -10.00 | best = -10 | epsilon = 0.086
Episodes 491-500 | avg = -10.00 | best = -10 | epsilon = 0.082
Episodes 501-510 | avg = -10.00 | best = -10 | epsilon = 0.078
Episodes 511-520 | avg = -10.00 | best = -10 | epsilon = 0.074
Episodes 521-530 | avg = -10.00 | best = -10 | epsilon = 0.070

Episodes 531-540	avg = -10.00	best = -10	epsilon = 0.067
Episodes 541-550	avg = -10.00	best = -10	epsilon = 0.063
Episodes 551-560	avg = -10.00	best = -10	epsilon = 0.060
Episodes 561-570	avg = -10.00	best = -10	epsilon = 0.057
Episodes 571-580	avg = -10.00	best = -10	epsilon = 0.055
Episodes 581-590	avg = -10.00	best = -10	epsilon = 0.052
Episodes 591-600	avg = -10.00	best = -10	epsilon = 0.050
Episodes 601-610	avg = -10.00	best = -10	epsilon = 0.050
Episodes 611-620	avg = -10.00	best = -10	epsilon = 0.050
Episodes 621-630	avg = -10.00	best = -10	epsilon = 0.050
Episodes 631-640	avg = -10.00	best = -10	epsilon = 0.050
Episodes 641-650	avg = -10.00	best = -10	epsilon = 0.050
Episodes 651-660	avg = -10.00	best = -10	epsilon = 0.050
Episodes 661-670	avg = -10.00	best = -10	epsilon = 0.050
Episodes 671-680	avg = -10.00	best = -10	epsilon = 0.050
Episodes 681-690	avg = -10.00	best = -10	epsilon = 0.050
Episodes 691-700	avg = -10.00	best = -10	epsilon = 0.050
Episodes 701-710	avg = -10.00	best = -10	epsilon = 0.050
Episodes 711-720	avg = -10.00	best = -10	epsilon = 0.050
Episodes 721-730	avg = -10.00	best = -10	epsilon = 0.050
Episodes 731-740	avg = -10.00	best = -10	epsilon = 0.050
Episodes 741-750	avg = -10.00	best = -10	epsilon = 0.050
Episodes 751-760	avg = -10.00	best = -10	epsilon = 0.050
Episodes 761-770	avg = -10.00	best = -10	epsilon = 0.050
Episodes 771-780	avg = -10.00	best = -10	epsilon = 0.050
Episodes 781-790	avg = -10.00	best = -10	epsilon = 0.050
Episodes 791-800	avg = -10.00	best = -10	epsilon = 0.050
Episodes 801-810	avg = -5.00	best = 40	epsilon = 0.050
Episodes 811-820	avg = -5.00	best = 40	epsilon = 0.050
Episodes 821-830	avg = -10.00	best = -10	epsilon = 0.050
Episodes 831-840	avg = -10.00	best = -10	epsilon = 0.050
Episodes 841-850	avg = -10.00	best = -10	epsilon = 0.050
Episodes 851-860	avg = -10.00	best = -10	epsilon = 0.050
Episodes 861-870	avg = -10.00	best = -10	epsilon = 0.050
Episodes 871-880	avg = -10.00	best = -10	epsilon = 0.050
Episodes 881-890	avg = -10.00	best = -10	epsilon = 0.050
Episodes 891-900	avg = -10.00	best = -10	epsilon = 0.050
Episodes 901-910	avg = -10.00	best = -10	epsilon = 0.050
Episodes 911-920	avg = -10.00	best = -10	epsilon = 0.050
Episodes 921-930	avg = -10.00	best = -10	epsilon = 0.050
Episodes 931-940	avg = -10.00	best = -10	epsilon = 0.050
Episodes 941-950	avg = -10.00	best = -10	epsilon = 0.050
Episodes 951-960	avg = -10.00	best = -10	epsilon = 0.050
Episodes 961-970	avg = -10.00	best = -10	epsilon = 0.050
Episodes 971-980	avg = -10.00	best = -10	epsilon = 0.050
Episodes 981-990	avg = -10.00	best = -10	epsilon = 0.050
Episodes 991-1000	avg = -10.00	best = -10	epsilon = 0.050

Understanding what is going wrong

From this history you can see a major issue in the neural network: the model is learning and performing well, however in the later iterations it seems to be underperforming such that it feels that it has started from scratch again and has not learnt anything. Such degradation in the Neural Network's performance can be due to a number of reasons as listed below. But from what I see, I think the neural network is adopting a bad policy after it scores high, this can be noticed in its behavioral changes during the under-performing episodes, where the model is seen either flying straight to the ground, or flying into the ceiling.

Neural Network Degradation

Overfitting

Data Drift

Learning a Bad Policy

Catastrophic Forgetting

How can we improve it

Resources

- Teach Neural Network to play snake game | Reinforcement Learning | Reward maximization | AI gameplay by Vizura: <https://www.youtube.com/watch?v=03zDwIzRPQI&t=11s>
- Understanding Neural Networks by Building One from Scratch: A Beginner's Journey: <https://medium.com/@pankajgoyal4152/understanding-neural-networks-by-building-one-from-scratch-a-beginners-journey-3a11617313a4>